

# Implementation Artifact Manifest

---

This note documents the implementation inspected at `/home/ubuntu/autonomous_agri_system/` for the Master's thesis.

## Repository composition

---

The repository is a Python/PyTorch prototype organized into `perception`, `planning`, `identification`, `utils`, `configs`, `tests`, and notebook directories. It contains modules for multimodal BEV perception, global and hybrid local planning, an MPC-like trajectory tracker, and crop-image semantic segmentation.

## Perception

---

`perception/bev_transformer.py` implements `BEVTransformer`. It accepts RGB camera tensors `(B, 3, H, W)` and rasterized LiDAR BEV tensors `(B, 1, H, W)`. Separate three-stage convolutional encoders map both modalities to 128-channel feature maps. The feature maps are flattened into tokens and fused using bidirectional eight-head `MultiheadAttention`; a `LayerNorm` is then applied to the average of both attended streams. A convolutional decoder produces occupancy logits, six-class semantic logits, and a confidence map, each interpolated to a 200 by 200 BEV grid, plus a six-dimensional detection output. The default perception configuration uses 2,000 synthetic samples, 256-pixel images, batch size 8, eight epochs, Adam learning rate  $3e-4$ , and random seed 42.

## Planning

---

`planning/global_planner.py` supplies the global coverage component. `planning/local_planner.py` generates a fixed lattice with 20 steps at 0.2 s, using three candidate speeds (0.8, 1.1, and 1.4) and five curvatures (-0.2 to 0.2). It scores candidates with a learned `TrajectoryScorerNet` and a classical cost that combines terminal distance to the goal with a heading-change penalty. The weighting is `alpha=0.65` for the learned score. All candidates are rejected if the inflated occupancy region (`safety_margin=2` cells) intersects an occupied cell. If none is safe, the planner returns a stationary emergency trajectory.

`planning/mpc_controller.py` implements an educational, solver-free sampled MPC-like tracker. It uses a kinematic bicycle model with 1.2 m wheelbase, 0.1 s discretization, 12-step horizon, steering limit 0.45 rad, and acceleration limit 1.2. At each cycle it evaluates a 9 by 7 grid of fixed steering and acceleration controls, rolling each candidate through the horizon and minimizing weighted position, heading, speed, and control-effort cost.

## Crop identification

---

`identification/deeplabv3plus.py` constructs a DeepLabV3+ semantic-segmentation model through `segmentation_models_pytorch`, using an ImageNet-pretrained EfficientNet-B4 encoder when available. It falls back to torchvision DeepLabV3 with ResNet-50 if the preferred dependency is unavailable.

## Verification status

---

The implementation contains unit tests for metrics, global planning, and local planning. This manifest records software inspection; it does not independently reproduce performance values or field-test claims.